

ВЕТВЛЕНИЕ — ЦИКЛЫ — ООП

Современные Технологии Программирования

Ахмад Али

Ветвление: if

if

```
if (condition) {  
    // Код, который будет выполнен, если условие истинно  
}
```

if/else

```
if (condition) {  
    // Код, который будет выполнен, если условие истинно  
} else {  
    // Код, который будет выполнен, если условие ложно  
}
```

If-else if-else

```
if (condition1) {  
    // Код  
} else if (condition2) {  
    // Код  
} else {  
    // Код, если ни одно из вышеуказанных условий не выполнено  
}
```

Тернарный оператор

```
dataType result = (condition) ? valueIfTrue : valueIfFalse;
```

Ветвление: switch

switch

```
switch (argument) {  
    case value1:  
        statement1;  
        break;  
    case value2:  
    case value3:  
        // Без break выполнение пойдёт к следующему case  
        statement2;  
        break;  
    case valueN:  
        statementN;  
        break;  
    default:  
        default_statement;  
        break;  
}
```

Switch с оператором -> (Java 14+)

```
switch (expression) {  
    case value1 -> statement1;  
    case value2, value3 -> statement2;  
    case valueN -> statementN;  
    default -> default_statement;  
}
```

switch как выражение с присваиванием результата

```
var result = switch (expression) {  
    case value1 -> result1;  
    case value2, value3 -> result2;  
    case valueN -> resultN;  
    default -> defaultResult;  
};
```

Ветвление: switch

switch как выражение с yield внутри блока (Java 14+)

```
var result = switch (expression) {  
    case value1 -> {  
        // statement1  
        // statement2  
        // .....  
        yield result1;  
    }  
    case value2, value3 -> {  
        // некоторая логика  
        yield result2;  
    }  
    default -> {  
        // statement1  
        yield defaultResult;  
    }  
};
```

switch с сопоставлением по типу (Java 17+)

```
switch (expression) {  
    case Type1 variable1 -> {  
        // код  
    }  
    case Type2 variable2 -> {  
        // код  
    }  
    case null -> {  
        // код  
    }  
    default -> {  
        // код  
    }  
}
```

Ветвление: switch

switch с сопоставлением по типу с оператором when (Java 21+)

```
switch (expression) {  
    case Type1 variable1 -> {  
        // код  
    }  
    case Type2 variable2 when condition -> {  
        // код  
    }  
    case null -> {  
        // код  
    }  
    default -> {  
        // код  
    }  
}
```

Ветвление



Циклы

while

```
while (condition) {  
    // тело цикла  
}
```

do-while

```
do {  
    // тело цикла.  
    // выполняется один  
    // раз как минимум.  
} while (условие);
```

Классический for

```
for (инициализация; условие; изменение) {  
    // тело цикла  
}
```

foreach (Расширенный for)

```
for (тип переменная : коллекция_или_массив)  
{  
    // тело цикла  
}
```

ЦИКЛЫ

break

- Прерывает текущий цикл (for, while, do-while, switch).
- Выход из цикла досрочно.
- Может использоваться с меткой для выхода из внешнего цикла.

continue

- Прерывает текущую итерацию цикла.
- Переходит сразу к следующей итерации (проверка условия и повтор).

Метки (Labels)

- Позволяют управлять вложенными циклами.
- Объявляются перед циклом:
labelName: while (...) { ... }
- Используются с break и continue
break label;
continue label;

Циклы



Базовые принципы объектно-ориентированного программирования

- Наследование
- Инкапсуляция
- Полиморфизм
- Абстракция

Наследование

- Наследование -механизм, который позволяет создавать новый класс/интерфейс на основе другого класса/интерфейса (родительского класса/интерфейса), и использовать свойства и методы родительского класса в соответствии с набором правил.
- Класс может наследовать только один класс, но реализовывать несколько интерфейсов.
- Ключевое слово **extends** используется, когда класс наследуется от класса или интерфейс наследуется от интерфейса, тогда как ключевое слово **implements** используется, когда класс реализует интерфейс.
- в Java если класс реализует интерфейс или наследует абстрактный класс, он обязан реализовать все абстрактные методы, за исключением:
 - методов с реализацией (default или static в интерфейсах),
 - если сам класс объявлен как abstract.

Наследование

- Ключевое слово `super`
 - `super()` - вызывает конструктор родительского класса
 - `super.methodName()` - вызывает метод родительского класса
 - `super.fieldName` - обращается к полю родительского класса
 - `InterfaceName.super.methodName()` - вызывает методы по умолчанию родительского интерфейса.
- Ключевое слово `this`
 - Используется внутри методов или конструкторов для ссылки на текущий экземпляр класса.
 - Как правило, применять `this` нужно в двух случаях:
 - Когда у переменной экземпляра класса и переменной метода/конструктора одинаковые имена;
 - Когда нужно вызвать конструктор одного типа (например, конструктор по умолчанию или параметризованный) из другого. Это еще называется явным вызовом конструктора.

Наследование

- Ключевое слово `final`
 - `final` переменная-значение нельзя изменить после присваивания.
 - `final` метод-метод нельзя переопределить в подклассе
 - `final` класс-класс нельзя наследовать
 - Если параметр метода объявлен как `final`, его нельзя переназначить внутри метода. Это не делает объект неизменяемым — только ссылку нельзя изменить.

Наследование

- Sealed-класс (запечатанный класс) (— это класс, который ограничивает список подклассов, которые могут его наследовать. Он даёт полный контроль над иерархией и повышает безопасность и предсказуемость кода.
- Чтобы указать разрешённые подклассы используется ключевое слово `permits`
- Подкласс запечатанного класса с модификатором `non-sealed` разрешает дальнейшее наследование от себя.

Наследование



Инкапсуляция

- Инкапсуляция -можно понять как механизм сокрытия внутренних деталей реализации объекта и предоставления внешнему миру только необходимых методов для взаимодействия с объектом, так и механизм, позволяющий объединить данные и методы, работающие с этими данными.
- Механизмы инкапсуляции в Java включают в себя:
 - Модификаторы доступа
 - Геттеры и сеттеры

Инкапсуляция

Модификаторы доступа

- `private`
- `protected`
- `public`
- Если не указать модификатор доступа, используется доступ по умолчанию, при котором члены класса доступны только внутри одного пакета.

Сеттеры

- Геттеры и сеттеры — это методы, которые позволяют безопасно получать и изменять значения полей объекта.
 - Getter (геттер) — возвращает значение поля
 - Setter (сеттер) — устанавливает значение поля

Тип видимости	Ключевое слово	Доступ			
		Свой класс	Свой пакет	Класс - наследник	Все классы
Закрытый	<code>private</code>	Есть	Нет	Нет	Нет
Пакет	(нет модификатора)	Есть	Есть	Нет	Нет
Защищенный	<code>protected</code>	Есть	Есть	Есть	Нет
Открытый	<code>public</code>	Есть	Есть	Есть	Есть

Инкапсуляция



Полиморфизм

- Полиморфизм-концепция, которая позволяет разным объектам выполнять одни и те же действия, но каждый по-своему в зависимости от его типа. Объекты обычно имеют общий интерфейс, обеспечивающий единый метод доступа.
- Полиморфизм времени компиляции (статический полиморфизм) достигается с помощью перегрузки методов.
- Полиморфизм времени выполнения (динамический полиморфизм) достигается с помощью переопределения методов.

Полиморфизм

- Переопределение метода (Overriding)-это изменение поведения метода суперкласса в подклассе с сохранением сигнатуры. Рекомендуется (но не обязательно) добавлять аннотацию @Override .
- Перегрузка (overloading) — это возможность в Java создавать несколько методов с одинаковым именем, но с разными параметрами в пределах одного класса.

Полиморфизм

- Оператор `instanceof` позволяет проверить, является ли объект экземпляром определённого класса или его подкласса.
- Восходящее преобразование (Upcasting) — это приведение от подкласса к суперклассу. Оно является автоматическим и безопасным.
- Нисходящее преобразование (Downcasting) — это приведение от суперкласса к подклассу. Оно требует явного указания и может вызвать `ClassCastException`, если объект не принадлежит нужному типу. Лучше предварительно проверить с помощью `instanceof`.

Полиморфизм



Абстракция

- Абстракция-процесс выделения существенных характеристик объекта и игнорирования мелких деталей, благодаря чему можно создавать модели объектов, содержащие только необходимую информацию и функциональные возможности, скрывая сложность реализации.
- Абстракция реализуется через абстрактные классы и интерфейсы.
- Они определяют, что должен делать объект, но не раскрывают, как именно он это делает.

Абстракция

